

S U D D E N A T T A C K T R A C K E R

SATrk

서든어택 전적 조회 · 분석 웹 서비스
프로젝트 설계 문서 패키지

기획서 · 요구사항 명세 · 테이블 명세 · 스토리보드 · 구조 명세 · 향후 제안

기반 API : NEXON Open API (서든어택)

Stack : Spring Boot 4 / Java 21 / MySQL · React 19 / Vite

작성일 : 2026-07-03 · Version 1.2

목차

1. 기획서 — 서비스 배경 · 목적 · 범위 · 기능 개요
2. 요구사항 명세서 — 기능(FR) · 비기능(NFR) · 외부 인터페이스
3. 테이블 명세서 — DB 스키마 · API 데이터 모델
4. 스토리보드 — 화면 흐름 · 화면별 구성/이벤트
5. 구조 명세서 — 시스템 아키텍처 · 패키지 · API · 데이터 흐름
6. 향후 개선 제안 — 추가하면 좋은 기능 · 기술 개선

본 문서는 **SATrk** 저장소의 실제 소스코드(backend Spring Boot, frontend React)와 guide/ 폴더의 API 스펙·구현 가이드를 근거로 역설계(reverse engineering)하여 작성되었습니다. 각 명세는 현재 구현된 코드 기준이며, 향후 계획 항목은 **계획** 으로 표기합니다. 소주제는 페이지 중간에 잘리지 않도록 배치되어 일부 여백이 발생할 수 있습니다.

기획서

1.1 프로젝트 개요

SATrk(SA-Tracker)는 넥슨의 FPS 게임 **서든어택** 이용자를 위한 전적 조회 및 전투 데이터 분석 웹 서비스이다. 사용자가 닉네임을 입력하면 NEXON Open API를 통해 해당 이용자의 기본 정보, 계급/티어, 최근 동향, 무기별 지표, 매치 목록 및 매치 상세 기록을 조회하여 한 화면에서 통합적으로 확인할 수 있도록 한다. OP.GG·전적검색 서비스의 서든어택 특화 버전을 지향한다.

항목	내용
서비스명	SATrk (Sudden Attack Tracker)
유형	웹 기반 게임 전적 조회/분석 서비스 (SPA + REST API)
대상 게임	서든어택 (NEXON)
데이터 출처	NEXON Open API — https://open.api.nexon.com
플랫폼	데스크톱 웹 브라우저 (반응형 고려)
핵심 기술	Backend: Spring Boot 4 / Java 21 / MySQL · Frontend: React 19 / Vite / React Router

1.2 추진 배경 및 목적

서든어택은 국내 대표 온라인 FPS이나, 게임 내 제공되는 전적 정보는 조회 편의성과 시각화 측면에서 제한적이다. NEXON이 서든어택 Open API를 공개하면서 외부 서비스가 전적 데이터를 활용할 수 있게 되었고, 이를 기반으로 다음 목적을 달성한다.

- **통합 조회** — 닉네임 한 번 입력으로 기본정보·계급·티어·최근동향·무기지표·매치기록을 한 화면에 통합 제공
- **전투 분석** — 킬/데스 추이, 승률, 무기 계열별(돌격·저격·특수) 지표를 요약·시각화하여 플레이 스타일 진단
- **매치 심층 분석** — 매치 상세에서 팀 구성·개인 성적(킬/데스/헤드샷/데미지)을 표로 확인, 맵별 필터 지원
- **즐거찾기** — 자주 조회하는 이용자를 저장하여 빠르게 재조회

1.3 타겟 사용자

사용자 유형	니즈	주요 사용 기능
일반 이용자	본인/상대 전적을 빠르게 확인하고 실력 가늠	검색, 프로필, 최근 동향
랭크/경쟁 유저	티어·계급 랭킹, 킬데스 추이로 성장 관리	계급/티어 요약, 매치 추이
클랜/팀 운영자	클랜원·상대 팀 전력 분석, 매치 복기	매치 상세, 팀 테이블, 즐겨찾기
스트리머/분석가	맵별·모드별 통계 기반 콘텐츠 제작	맵 필터, 매치 요약

1.4 핵심 기능 (기능 맵)

영역	기능	설명
검색	닉네임 검색	홈 검색창/네비게이션에서 닉네임 입력 → 플레이어 페이지 이동
	즐거찾기 재검색	저장된 즐거찾기 목록에서 원클릭 조회
프로필	기본 정보	닉네임·계정생성일·칭호·클랜·매너등급
	계급/티어 요약	통합/시즌 계급·랭킹, 솔로/파티 랭크 티어·점수 (이미지 포함)
	최근 동향	최근 승률·킬데스·무기 계열 킬데스율
	전투/무기 요약	승률, 무기 계열(돌격·저격·특수) 비중 요약
매치	매치 목록	모드·유형·맵·범위(최근/전체) 필터, 페이지네이션
	매치 상세	팀별 개인 기록(킬·데스·헤드샷·데미지·어시스트) 테이블
	매치 요약	주 플레이 모드/유형, 플레이 스타일, 킬데스 추이
	맵별 조회	특정 맵 기준 매치 상세 목록
랭킹	랭킹 조회	계급/시즌 계급 랭킹, 랭크전 티어·점수 통합 조회
즐거찾기	추가/조회/삭제	즐거찾기 이용자 CRUD (로컬 DB 저장)

1.5 서비스 범위

포함 (In-Scope)

- NEXON Open API 연동 전적 조회 전 기능
- 프로필/매치/랭킹/무기/통계 화면
- 즐거찾기 저장(MySQL)
- API 호출 최적화(캐시·레이트리밋·재시도)

미포함 (Out-of-Scope)

계획

- 회원가입/로그인·개인화 계정
- 실시간 알림·구독
- 결제·유료 기능
- 모바일 네이티브 앱

1.6 기술 스택

구분	기술
Backend	Spring Boot 4.0.6, Java 21, Spring Web(MVC), Spring Data JPA, Bean Validation, Lombok
Database	MySQL 8 (JDBC, ddl - auto=update), 스키마 satrk
외부 API	NEXON Open API (서든어택) — RestTemplate 연동
Frontend	React 19, Vite 8, React Router 7, Axios
빌드/실행	Gradle (backend, port 8085), npm/Vite dev server (frontend)

1.7 마일스톤 추정

단계	산출물	비고
M1 설계	API 스펙 정리, DB 설계, 화면 설계	guide 문서
M2 백엔드	NEXON 클라이언트, 도메인별 서비스/컨트롤러	완료 기준 코드 존재
M3 프론트	홈/플레이어/매치 페이지, 공통 컴포넌트	완료 기준 코드 존재
M4 안정화	캐시·레이트리밋·에러 처리	진행/보완
M5 배포	운영 환경 구성, 성능 튜닝	계획

1.8 리스크 및 대응

리스크	대응 방안
NEXON API 호출 제한(429)	최소 호출 간격 350ms 강제, 최대 3회 재시도, OUID 6시간 캐시
데이터 지연(게임 종료 후 ~10분)	UI에 데이터 갱신 지연 안내, 조회 시점 표기
조회 기간 제한(2025-01-24 이후)	기간 외 데이터 없음 안내 처리
API Key 노출	서버 측 보관, 프론트 미노출 (프록시 방식)
메타(계급/티어 이미지) 변경	NexonMetaCacheService로 메타 캐싱 및 매핑

요구사항 명세서

2.1 기능 요구사항 (Functional Requirements)

ID	기능	요구사항 설명	우선순위
FR-01	플레이어 검색	사용자는 닉네임(키워드)으로 이용자를 검색할 수 있다. 검색 성공 시 플레이어 페이지로 이동한다.	필수
FR-02	플레이어 프로필 조회	닉네임으로 기본정보·계급·티어·최근동향과 관련 이미지(계급/티어)를 통합 조회한다.	필수
FR-03	랭킹 조회	통합/시즌 계급·랭킹, 솔로/파티 랭크전 티어·점수를 조회한다.	필수
FR-04	통계 조회	최근 승률, 킬데스율, 무기 계열(돌격·저격·특수) 킬데스율을 조회한다.	권장
FR-05	무기 통계 조회	무기 계열별 비중(돌격·저격·특수)을 조회한다.	권장
FR-06	매치 목록 조회	닉네임 기준 매치 목록을 조회한다. 페이지·범위(최근·전체)·매치유형·모드·맵으로 필터링한다.	필수
FR-07	매치 상세 조회	매치 ID로 팀 구성 및 참가자별 킬·데스·헤드샷·데미지·어시스트를 조회한다.	필수
FR-08	매치 요약	주 플레이 모드/유형, 플레이 스타일, 킬데스 추이(추세) 및 요약 항목을 제공한다.	권장
FR-09	맵별 매치 조회	특정 맵 이름 기준으로 매치 상세 목록을 조회한다.	권장
FR-10	즐거찾기 추가	닉네임을 즐겨찾기에 저장한다.	권장
FR-11	즐거찾기 목록	저장된 즐겨찾기 목록을 조회한다.	권장
FR-12	즐거찾기 삭제	ID로 즐겨찾기 항목을 삭제한다.	권장
FR-13	인트로 연출	최초 방문 시 로딩 인트로를 노출하고, 세션 내 재노출하지 않는다(sessionStorage).	선택

2.2 비기능 요구사항 (Non-Functional Requirements)

ID	분류	요구사항 설명
NFR-01	성능/처리량	NEXON API 호출 간 최소 간격 350ms를 유지하여 호출 제한(429)을 예방한다.
NFR-02	안정성	외부 API 실패 시 최대 3회 재시도하며, 실패 시 사용자에게 오류 메시지를 반환한다.
NFR-03	캐싱	닉네임→UUID 매핑은 6시간 캐시하여 중복 호출을 줄인다. 메타(계급/티어 이미지)는 캐시한다.
NFR-04	보안	NEXON API Key는 서버 환경설정에 보관하고 프론트엔드에 노출하지 않는다(서버 프록시).
NFR-05	확장성	도메인별 패키지(player, match, ranking, stats, weapon, favorite, gameMap)로 분리하여 기능 추가가 용이하다.
NFR-06	사용성	로딩/오류 상태를 화면에 명시(“불러오는 중…”, 오류 메시지)하고, 필터·페이지네이션을 제공한다.
NFR-07	호환성	Frontend(5173)–Backend(8085) 간 CORS 허용(WebConfig)으로 로컬 개발 연동을 보장한다.
NFR-08	이식성	MySQL 스키마는 ddl-auto=update로 자동 반영되어 초기 구동이 단순하다.
NFR-09	국제화/시간	날짜는 UTC로 수신(...Z), 서버 타임존 Asia/Seoul 기준 표기한다.

2.3 외부 인터페이스 요구사항 (NEXON Open API 의존)

항목	내용
Base URL	https://open.api.nexon.com
인증	HTTP 헤더 x-nxopen-api-key 에 API KEY 전달
데이터 제공 지연	게임 종료 후 평균 10분
조회 가능 기간	2025년 1월 24일 이후 데이터
식별자(oid)	게임 콘텐츠 변경 시 변경될 수 있음 (캐시 갱신 필요)
에러 코드	400 Bad Request · 403 Forbidden · 429 Too Many Requests · 500 Internal Server Error
매치 조회 한도	최대 1,000개

연동 NEXON API 엔드포인트

NEXON Endpoint	용도
GET /suddenattack/v1/id	닉네임 → 계정 식별자(oid)
GET /suddenattack/v1/user/basic	기본 정보(칭호·클랜·매너등급 등)
GET /suddenattack/v1/user/rank	통합/시즌 계급·경험치·랭킹
GET /suddenattack/v1/user/tier	솔로/파티 랭크전 티어·점수
GET /suddenattack/v1/user/recent-info	최근 승률·킬데스·무기 계열율
GET /suddenattack/v1/match	매치 목록(모드·유형 필터)
GET /suddenattack/v1/match-detail	매치 상세(팀·개인 기록)
GET /static/suddenattack/meta/logo	로고 이미지
GET /static/suddenattack/meta/grade	통합 계급 이미지 목록
GET /static/suddenattack/meta/season_grade	시즌 계급 이미지 목록
GET /static/suddenattack/meta/tier	티어 이미지 목록

2.4 데이터/입력 규칙

- **match_result** : "1"=승리, "2"=패배, "3"=무승부. '진짜를 모아라' 모드는 승패 미기록.
- **정렬 규칙** : 개인전·'진짜를 모아라'는 킬 수 높은 순 → 동일 시 데스 적은 순.
- **clan_name 노출** : 클랜전·퀵매치 클랜전·클랜 랭크전에서만 노출.
- **매치 모드** : 개인전 / 데스매치 / 폭파미션 / 진짜를 모아라.
- **매치 유형** : 일반전 / 클랜전 / 퀵매치 클랜전 / 클랜 랭크전 / 랭크전 솔로 / 랭크전 파티 / 토너먼트.

테이블 명세서

SATrk은 대부분의 데이터를 NEXON API에서 실시간 조회하며, 로컬 MySQL에는 **즐거찾기**와 **클랜원** 정보만 영속화한다. 본 장은 (A) 물리 DB 테이블과 (B) API 응답 데이터 모델(논리 스키마)을 함께 명세한다.

3.1 물리 DB 테이블

① favorite — 즐겨찾기 (JPA Entity: Favorite)

컬럼	타입	제약	설명
id	BIGINT	PK, AUTO_INCREMENT	즐거찾기 고유 식별자
user_name	VARCHAR(255)	—	서든어택 닉네임

매핑 근거: favorite/entity/Favorite.java — @Id @GeneratedValue(IDENTITY) Long id; String userName;.ddl-auto=update 설정으로 애플리케이션 기동 시 자동 생성된다.

② clan_members — 클랜원 [가이드 설계](#)

컬럼	타입	제약	설명
id	BIGINT	PK, AUTO_INCREMENT	고유 식별자
user_name	VARCHAR(50)	NOT NULL, UNIQUE	서든어택 닉네임
clan_name	VARCHAR(50)	DEFAULT '모'	클랜명
oid	VARCHAR(200)	—	NEXON API oid (캐시용)
created_at	DATETIME	DEFAULT NOW()	생성 시각
updated_at	DATETIME	ON UPDATE NOW()	수정 시각

출처: guide/지식2_구현가이드.md의 DB 설계 예시. 현재 JPA 엔티티로는 미구현이며 향후 클랜 분석 기능 확장 시 사용.

3.2 API 응답 데이터 모델 (논리 스키마 / DTO)

UserBasicDto — 기본 정보

필드	타입	설명
user_name	String	닉네임
user_date_create	String(UTC)	계정 생성일
title_name	String	장착 칭호명
clan_name	String	소속 클랜명
manner_grade	String	매너 등급

UserRankDto — 계급 정보

필드	타입	설명
grade	String	통합 계급
grade_exp	Integer	통합 계급 경험치
grade_ranking	Integer	통합 계급 랭킹
season_grade	String	시즌 계급
season_grade_exp	Integer	시즌 계급 경험치
season_grade_ranking	Integer	시즌 계급 랭킹

UserTierDto & UserRecentInfoDto

UserTierDto 필드	설명	UserRecentInfoDto 필드	설명
solo_rank_match_tier	솔로 랭크전 티어	recent_win_rate	최근 승률
solo_rank_match_score	솔로 랭크전 점수	recent_kill_death_rate	최근 킬데스
party_rank_match_tier	파티 랭크전 티어	recent_assault_rate	돌격소총 킬데스
party_rank_match_score	파티 랭크전 점수	recent_sniper_rate	저격소총 킬데스
		recent_special_rate	특수총 킬데스

MatchDto — 매치 목록 항목

필드	타입	설명
match_id	String	매치 식별자
match_type	String	매치 유형(일반전/클랜전/랭크전 등)
match_mode	String	매치 모드(개인전/폭파미션 등)
date_match	String(UTC)	매치 일시
match_result	String	1=승 · 2=패 · 3=무
kill / death / assist	Integer	킬 / 데스 / 어시스트

MatchDetailItemDto — 매치 상세(참가자별 기록)

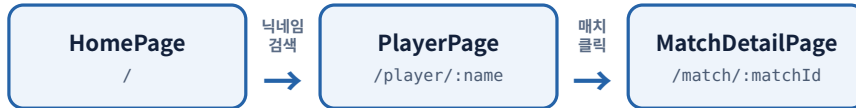
필드	타입	설명
team_id	String	팀 식별자
match_result	String	승/패 결과
user_name	String	참가자 닉네임
season_grade	String	시즌 계급
clan_name	String	클랜명(클랜전 계열만)
kill / death / assist	Integer	킬 / 데스 / 어시스트
headshot	Integer	헤드샷 수
damage	Double	누적 데미지

3.3 서버 응답 DTO (프론트 계약)

응답 DTO	주요 필드
PlayerResponseDto	userName, ould, basic, rank, tier, recent, images
RankingResponseDto	grade, gradeRanking, seasonGrade, seasonGradeRanking, solo/partyRankMatchTier · Score
StatsResponseDto	winRate, killDeathRate, assaultRate, sniperRate, specialRate
WeaponStatsResponseDto	assaultRate, sniperRate, specialRate
MatchListResponseDto	userName, page, size, totalCount, matches[]
MatchDetailResponseDto	matchId, matchType, matchMode, dateMatch, matchMap, matchDetail[]
MatchSummaryResponseDto	primaryMode, primaryType, playStyle, killDeathTrend[], summaries[]
GameMapResponseDto	userName, mapName, matches[] (MatchDetailResponseDto)
FavoriteResponseDto	id, userName
ImagesDto	계급/시즌계급/티어/로고 이미지 URL 묶음

스토리보드

4.1 화면 흐름도 (Screen Flow)

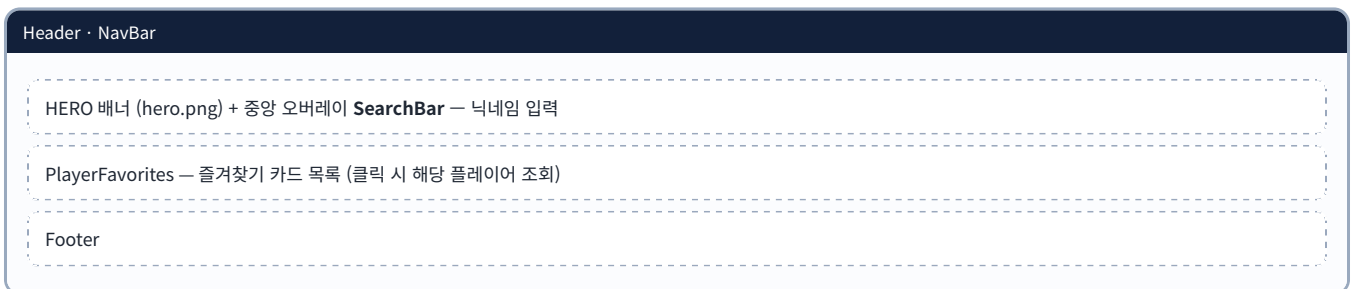


- 즐겨찾기 클릭 : HomePage → PlayerPage 로 바로 이동
- 매치 검색/필터 : MatchDetailPage 내에서 목록 갱신 후 상세 선택
- 최초 방문 : LoadingIntro 재생 → 세션 저장(sessionStorage) 후 재방문 시 스킵

4.2 라우팅 정의

경로	컴포넌트	설명
/	HomePage	메인 · 검색 · 즐겨찾기 진입점
/player/:name	PlayerPage	플레이어 프로필/요약 종합
/match	MatchDetailPage	매치 아카이브(검색·필터)
/match/:matchId	MatchDetailPage	특정 매치 상세 진입

4.3 화면별 상세 — S-01 HomePage



구성요소	이벤트	동작
LoadingIntro	최초 진입	인트로 재생 → sessionStorage['hasSeenIntro'] 저장 후 스킵
SearchBar	닉네임 입력 후 검색	/player/:name 로 이동
PlayerFavorites	카드 클릭 / 삭제	GET /api/favorite · DELETE /api/favorite/{id}

4.4 화면별 상세 — S-02 PlayerPage

Header · NavBar	
PlayerProfile — 닉네임·칭호·클랜·계정정보 + 계급/티어 이미지	
RankSummary — 통합/시즌 계급·랭킹, 랭크 티어·점수	WeaponSummary — 돌격/저격/특수 비중
CombatSummary — 승률·킬데스 등 전투 지표	MatchSummary — 주 모드/유형·플레이스타일·킬데스 추이
Footer	

상태	표기
로딩	“불러오는 중...”
오류	오류 메시지(player-error) — 존재하지 않는 닉네임 등
정상	record-banner + 각 요약 컴포넌트 렌더

데이터: 진입 시 GET /api/player?userName=:name 1회 호출. 각 요약은 필요 시 stats/weapon/ranking/match-summary API 사용.

4.5 화면별 상세 — S-03 MatchDetailPage

Header · NavBar	
MATCH DATA 배너 + 닉네임 검색창	
MatchFilter — 범위(최근/전체)·매치유형·모드·맵 선택	
MatchCard 목록 — 매치별 카드(승/패·킬데스), 클릭 시 상세 로드 + 페이지네이션	TeamTable — 선택 매치의 팀별 개인 기록 (Normal / Clan 테이블 분기)
Footer	

이벤트	API	동작
닉네임 검색/필터 적용	GET /api/match	목록 갱신 (page·scope·matchType·matchMode·matchMap)
매치 카드 선택	GET /api/match/detail	상세 로드 (detailCache로 중복 호출 방지)
페이지 이동	GET /api/match	page 파라미터 변경

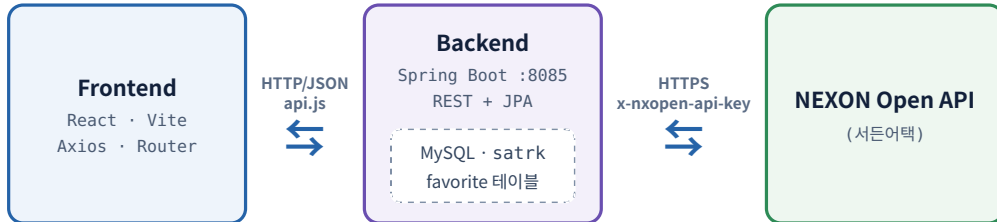
TeamTable은 매치 유형이 클랜 계열이면 ClanTeamTable(클랜명 노출), 그 외 NormalTeamTable로 렌더한다.

4.6 공통 UI 요소

컴포넌트	역할
Header	상단 로고/브랜드 영역
NavBar	전역 내비게이션 및 검색 진입
Footer	하단 정보/출처(NEXON Open API)
SearchBar	닉네임 검색 입력 및 이동
LoadingIntro	최초 방문 연출

구조 명세서

5.1 시스템 아키텍처



프론트엔드는 NEXON API를 직접 호출하지 않고 항상 백엔드를 경유(프록시)한다. 백엔드는 NexonApiClient를 통해 외부 API를 호출하고, 캐시·레이트리밋·재시도를 담당하며, 즐겨찾기만 MySQL에 저장한다.

5.2 백엔드 패키지 구조 (com.sa.trk)

```

com.sa.trk
├── BackendApplication.java      (main)
├── config/                     NexonProperties, WebConfig(CORS)
├── common/dto/                 ImagesDto
├── nexon/                      client(NexonApiClient) · dto(원천 API DTO) · service(NexonMetaCacheService)
├── player/                    controller · service · dto(PlayerResponseDto)
├── search/                    controller · service → PlayerService 위임
├── ranking/                   controller · service · dto
├── stats/                     controller · service · dto
├── weapon/                    controller · service · dto
├── match/                     controller · service · dto(List/Detail/Summary/Trend/Item)
├── gameMap/                   controller · service · dto → MatchService 재사용
└── favorite/                  controller · service · repository · entity · dto
  
```

도메인별 계층형 구조(Controller → Service → NexonApiClient/Repository). search · ranking · stats · weapon · gameMap 서비스는 자체 API 호출 대신 **PlayerService / MatchService**를 재사용하여 데이터를 가공한다.

5.3 REST API 목록 (백엔드)

Method	경로	파라미터 · 응답
GET	/api/search	keyword → PlayerResponseDto
GET	/api/player	userName → PlayerResponseDto
GET	/api/ranking	userName → RankingResponseDto
GET	/api/stats	userName → StatsResponseDto
GET	/api/weapon	userName → WeaponStatsResponseDto
GET	/api/match	userName, page=1, scope=RECENT, matchType, matchMode=ALL, matchMap=ALL → MatchListResponseDto
GET	/api/match/detail	matchId → MatchDetailResponseDto
GET	/api/match/summary	userName → MatchSummaryResponseDto
GET	/api/map	userName, mapName, scope=RECENT, matchType, page=1 → GameMapResponseDto
POST	/api/favorite	userName → FavoriteResponseDto
GET	/api/favorite	→ List<FavoriteResponseDto>
DELETE	/api/favorite/{id}	id → void

5.4 핵심 데이터 흐름 — 플레이어 조회

```
SearchBar → GET /api/player?userName=닉
└─ PlayerService.getPlayer()
    1) NexonApiClient.getOuid(닉)      ← 6h 캐시 확인 / 미스 시 /v1/id 호출
    2) getUserBasic(ouid)              → /v1/user/basic
    3) getUserRank(ouid)               → /v1/user/rank
    4) getUserTier(ouid)               → /v1/user/tier
    5) getUserRecentInfo(ouid)         → /v1/user/recent-info
    6) NexonMetaCacheService 로 계급/티어 이미지 매핑 → ImagesDto
└─ PlayerResponseDto 조립 후 반환
(각 외부 호출은 최소 350ms 간격 · 실패 시 최대 3회 재시도)
```

5.5 NexonApiClient — 호출 정책

정책	값 / 설명
최소 호출 간격	MIN_REQUEST_INTERVAL_MS = 350ms (동기화 처리로 429 예방)
재시도	MAX_RETRY_COUNT = 3 (HttpClientErrorException 대응)
OUID 캐시	OUID_CACHE_DURATION = 6시간, ConcurrentHashMap · 닉네임 소문자 키
인증 헤더	x-nxopen-api-key (NexonProperties에서 키 주입)
메타 캐시	NexonMetaCacheService — 계급/시즌계급/티어/로고 이미지 매핑

5.6 프론트엔드 구조

```
frontend/src
├─ main.jsx · App.jsx          (BrowserRouter 라우팅)
├─ api/api.js                  (axios · baseURL http://localhost:8085)
├─ pages/                      HomePage · PlayerPage · MatchDetailPage
├─ components/
│   ├── Header · NavBar · Footer · SearchBar · LoadingIntro · PlayerFavorites
│   ├── MatchCard · MatchFilter
│   ├── TeamTable(Normal/Clan)
│   └── PlayerProfile(RankSummary · WeaponSummary · MatchSummary · CombatSummary · MatchForm)
├─ styles/global.css · index.css · App.css
└─ assets/ hero.png 등
```

5.7 환경 구성

항목	값
Backend 포트	8085
DB URL	jdbc:mysql://localhost:3306/satrk?serverTimezone=Asia/Seoul
JPA	ddl-auto=update · show-sql=true
NEXON Base URL	https://open.api.nexon.com
Frontend Dev	Vite (기본 5173) · Axios baseURL 8085
CORS	WebConfig에서 프론트 오리진 허용

비고 — 본 구조 명세는 저장소의 실제 소스(컨트롤러·서비스·DTO·엔티티)와 application.properties, NexonApiClient.java를 근거로 작성되었습니다. 운영 배포 시 API Key·DB 비밀번호는 환경변수/시크릿으로 분리 권장합니다.

향후 개선 제안

현재 구현된 SATrk을 기준으로, 사용자 가치와 서비스 완성도를 높이기 위해 추가하면 좋을 항목을 우선순위와 함께 정리한다. **상** 은 사용자 체감·핵심 가치가 큰 항목, **중** 은 차별화 요소, **하** 는 운영 성숙도 항목이다.

6.1 기능 강화

제안	내용	우선
데이터 시각화	킬데스 추이·승률을 실제 그래프(Chart.js/Recharts)로 렌더링. killDeathTrend 데이터를 라인/영역 차트로 표현하면 분석 가치가 크게 상승.	상
플레이어 비교	두 명 이상 닉네임을 나란히 비교(승률·킬데스·티어). 라이벌/친구 대결 콘텐츠로 활용.	중
검색 편의	닉네임 자동완성·최근 검색어·오타 보정. 즐겨찾기와 연계해 진입 마찰 감소.	상
즐거찾기 확장	메모·태그·정렬, 즐겨찾기별 최근 전적 미리보기. 현재는 닉네임만 저장.	중
클랜 분석	가이드에 설계된 clan_members를 활용해 클랜원 일괄 전적·클랜 랭킹·내전 분석 제공.	중
맵/모드 통계	맵별 승률·선호 모드 히트맵, 시간대별 플레이 패턴 등 집계 지표.	하

6.2 데이터 · 분석

제안	내용	우선
매치 로컬 적재	조회한 매치를 MySQL에 누적 저장하여 API 한도(1000개)·기간(2025-01-24~) 제약을 넘어 장기 추세 분석. 배치 수집 스케줄러 도입.	상
지표 캐싱 계층	Redis 등으로 플레이어·매치 응답을 단기 캐시하여 재조회 지연·API 호출 절감(현재는 OUID만 6h 캐시).	중
파생 통계	헤드샷 비율·라운드당 데미지·클러치 등 원천 데이터로 계산 가능한 심화 지표 산출.	하

6.3 사용자 경험 (UX)

제안	내용	우선
반응형·모바일	모바일 뷰포트 대응 및 PWA(홈 추가·오프라인 캐시)로 접근성 확대.	상
스켈레톤 로딩	“불러오는 중...” 텍스트 대신 스켈레톤 UI로 체감 대기시간 감소.	중
다크모드·테마	게임 톤에 맞춘 다크 테마 및 사용자 선호 저장.	하
공유 기능	플레이어/매치 결과 이미지·링크 공유(OG 태그, 카드 이미지 생성).	중
접근성·SEO	시맨틱 마크업·a11y 개선, 플레이어 페이지 메타 태그로 검색 노출 강화.	하

6.4 기술 · 운영

제안	내용	우선
시크릿 분리	NEXON API Key · DB 비밀번호를 환경변수/Vault로 분리(현재 application.properties에 평문). 저장소 노출 위험 제거.	상
전역 예외 처리	@ControllerAdvice로 NEXON 4xx·5xx·429를 표준 오류 응답으로 변환하여 프론트 처리 일관화.	상
테스트 보강	현재 BackendApplicationTests 1개뿐. 서비스·클라이언트 단위 테스트, API 계약 테스트 추가.	중
모니터링·로깅	구조적 로깅·요청 추적, 오류 알림(Sentry 등), API 호출량/응답시간 대시보드.	중
CI/CD·배포	Docker 컨테이너화, GitHub Actions 빌드·테스트·배포 파이프라인, 운영/개발 프로파일 분리.	중
레이트리미트 고도화	단일 350ms 직렬화 대신 토큰버킷·비동기 큐로 처리량 개선 및 다중 사용자 대응.	하

6.5 우선 적용 로드맵 제안

순서	항목	이유
1차 (즉시)	시크릿 분리 · 전역 예외 처리 · 데이터 시각화	보안 위험 제거와 사용자 체감 가치가 가장 큼
2차 (단기)	매치 로컬 적재 · 검색 자동완성 · 반응형/모바일	서비스 지속성·유입·재방문 개선
3차 (중기)	플레이어 비교 · 클랜 분석 · 캐싱 계층 · CI/CD	차별화 기능과 운영 성숙도 확보